

AKLAN DRUGSTORE

Inventory Management System

Intermediate Mobile Programming
Task Performance 1

Developer	Arjunren Valdez
Technology	.NET MAUI and C#
Platforms	Windows and Android
Project type	Simple local student application
Date	October 4, 2026

Developed by Arjunren Valdez

1. Project Overview

This application is a simple inventory management system for a local drugstore in Aklan, Panay Island. It reduces manual inventory checking by giving employees one place to view product quantities, update records, see low-stock information, and review transactions.

The implementation uses one .NET MAUI codebase for Windows and Android. Product and audit records are stored as JSON files in the device application-data folder, so the classroom demo does not need a database server or an internet connection.

Feature checklist

Requirement	Implementation	Status
Login / Logout	Local demo account with visible validation	Complete
Menu screen	Dashboard, Inventory, Audit Trail navigation	Complete
Inventory management	Add, update, delete, select, and display	Complete
Data visualization	Totals, low-stock count, and stock bar chart	Complete
Audit trail	Login, logout, add, update, and delete records	Complete
Windows / Android	Single .NET MAUI project built for both targets	Complete

Demo account

Username: **admin**

Password: **admin123**

Security note: the fixed account is only for a student demonstration. A real deployment should use password hashing, role-based access, secure key storage, and an online backup.

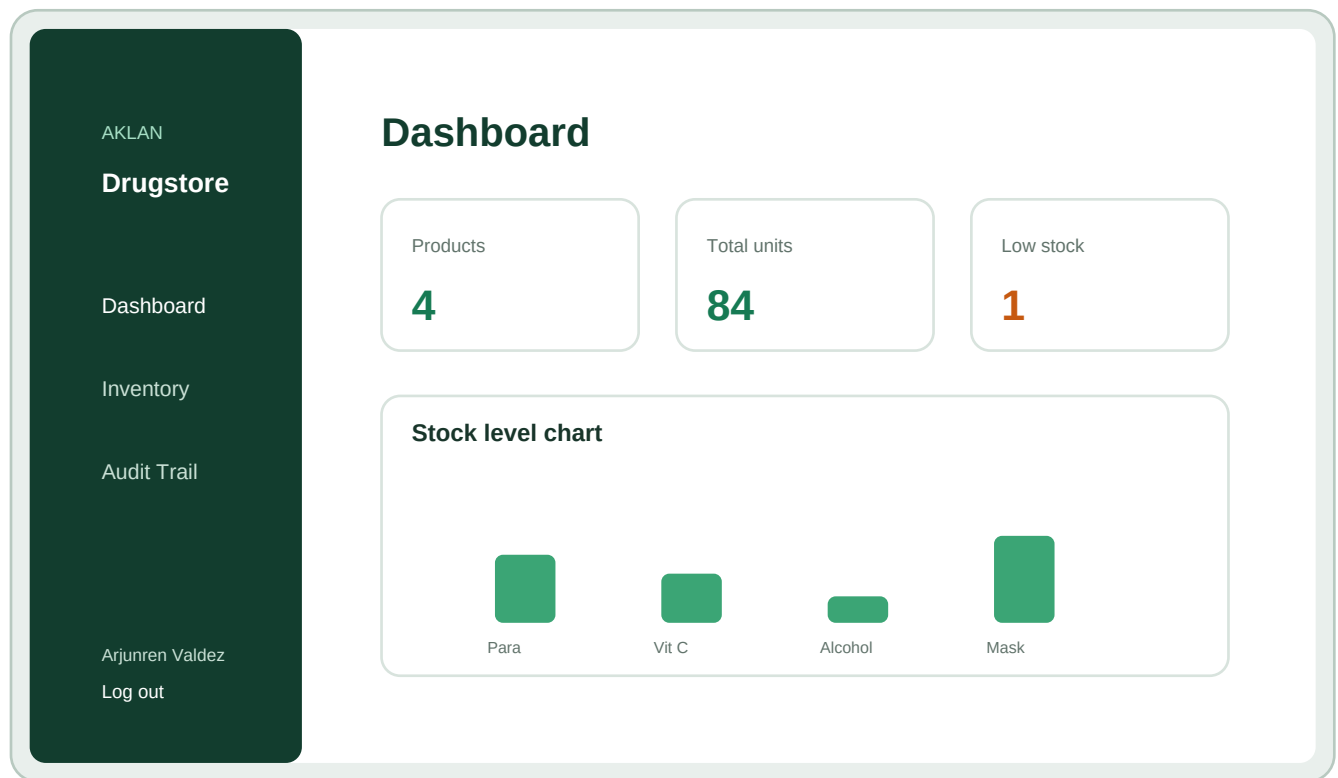
2. Interface Mockups

The interface uses a clean green drugstore theme. These mockups match the layout implemented in MainPage.xaml and show the intended Windows presentation.

Login screen

The login screen is a Windows-style application window with a light gray border and rounded corners. Inside the window is a white rectangular area containing the login form. At the top of the form is a green square icon with the white text 'Rx'. Below the icon is the title 'Aklan Drugstore' in bold black text, followed by the subtitle 'Inventory Management System' in a smaller black font. There are two input fields: the first is labeled 'Username: admin' and the second is labeled 'Password: *****'. Below the password field is a green rectangular button with the white text 'LOG IN'.

Dashboard and data visualization



The dashboard summarizes the number of products, total quantity, and low-stock items. A simple bar chart compares the current quantity of each product.

Inventory management

AKLAN

Drugstore

Dashboard

Inventory

Audit Trail

Arjunren Valdez

Log out

Inventory Management

Product details

Add

Update

Delete

Clear

Product	Category	Qty	Price
Paracetamol	Medicine	25	5.50
Vitamin C	Vitamins	18	8.00
Alcohol 70%	First Aid	9	45.00
Face Mask	Supplies	32	3.00

Employees can enter product details and use Add, Update, Delete, or Clear. Selecting a row loads its values into the form for editing. Validation prevents blank names, blank categories, negative quantities, and negative prices.

Audit trail

AKLAN

Drugstore

Dashboard

Inventory

Audit Trail

Arjunren Valdez

Log out

Audit Trail

Date and time	Action	Details
Oct 04, 01:22	UPDATE	Updated Alcohol 70%
Oct 04, 01:20	ADD	Added Face Mask
Oct 04, 01:18	LOGIN	Administrator logged in

The audit trail places the newest record first and records login, logout, add, update, and delete actions with their date, time, and details.

3. How to Open and Run

Step	Instruction
1	Open AklanDrugstoreInventory.slnx in Visual Studio 2026.
2	Confirm that the .NET MAUI workload and .NET 10 SDK are installed.
3	Choose Windows Machine or an Android emulator as the debug target.
4	Press F5 and wait for the application to start.
5	Log in using admin / admin123.
6	Use the menu to open Dashboard, Inventory, or Audit Trail.

Build commands

```
dotnet build AklanDrugstoreInventory.csproj -f net10.0-windows10.0.19041.0
dotnet build AklanDrugstoreInventory.csproj -f net10.0-android
```

Important files

MainPage.xaml contains the visible screens. MainPage.xaml.cs contains login, menu navigation, inventory CRUD, validation, dashboard totals, JSON persistence, and audit logic. The Models folder contains InventoryItem and AuditEntry. AppShell.xaml provides the application shell.

4. Source Code

The following pages contain the complete core source used by the project. Standard files generated by the .NET MAUI template remain in the repository, including platform manifests, font resources, icons, splash artwork, and default resource dictionaries.

Project File - AklanDrugstoreInventory.csproj

Source file: AklanDrugstoreInventory.csproj

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFrameworks>net10.0-android</TargetFrameworks>
    <TargetFrameworks Condition="!$([MSBuild]::IsOSPlatform('linux'))">$(TargetFrameworks);net10.0-ios;net10.0-maccatalyst</TargetFrameworks>
    <TargetFrameworks Condition="$([MSBuild]::IsOSPlatform('windows'))">$(TargetFrameworks);net10.0-windows10.0.19041.0</TargetFrameworks>
  </PropertyGroup>

  <!-- Note for MacCatalyst:
  The default runtime is maccatalyst-x64, except in Release config, in which case the default is maccatalyst-x64;maccatalyst-arm64.
  When specifying both architectures, use the plural <RuntimeIdentifiers> instead of the singular <RuntimeIdentifier>.
  The Mac App Store will NOT accept apps with ONLY maccatalyst-arm64 indicated;
  either BOTH runtimes must be indicated or ONLY macatalyst-x64. -->
  <!-- For example: <RuntimeIdentifiers>maccatalyst-x64;maccatalyst-arm64</RuntimeIdentifiers> -->

  <OutputType>Exe</OutputType>
  <RootNamespace>AklanDrugstoreInventory</RootNamespace>
  <UseMaui>true</UseMaui>
  <SingleProject>true</SingleProject>
  <ImplicitUsings>enable</ImplicitUsings>
  <Nullable>enable</Nullable>

  <!-- Display name -->
  <ApplicationTitle>Aklan Drugstore Inventory</ApplicationTitle>

  <!-- App Identifier -->
  <ApplicationId>com.arjunren.aklandrugstoreinventory</ApplicationId>

  <!-- Versions -->
  <ApplicationDisplayVersion>1.0</ApplicationDisplayVersion>
  <ApplicationVersion>1</ApplicationVersion>

  <!-- To develop, package, and publish an app to the Microsoft Store, see: https://aka.ms/MauiTemplateUnpackaged -->
  <WindowsPackageType>None</WindowsPackageType>

  <SupportedOSPlatformVersion Condition="$([MSBuild]::GetTargetPlatformIdentifier('$(TargetFramework)')) == 'ios'">15.0</SupportedOSPlatformVersion>
  <SupportedOSPlatformVersion Condition="$([MSBuild]::GetTargetPlatformIdentifier('$(TargetFramework)')) == 'maccatalyst'">15.0</SupportedOSPlatformVersion>
  <SupportedOSPlatformVersion Condition="$([MSBuild]::GetTargetPlatformIdentifier('$(TargetFramework)')) == 'android'">21.0</SupportedOSPlatformVersion>
  <SupportedOSPlatformVersion Condition="$([MSBuild]::GetTargetPlatformIdentifier('$(TargetFramework)')) == 'windows'">10.0.17763.0</SupportedOSPlatformVersion>
  <TargetPlatformMinVersion Condition="$([MSBuild]::GetTargetPlatformIdentifier('$(TargetFramework)')) == 'windows'">10.0.17763.0</TargetPlatformMinVersion>
</PropertyGroup>

  <ItemGroup>
    <!-- App Icon -->
    <MauiIcon Include="Resources\AppIcon\appicon.svg" ForegroundFile="Resources\AppIcon\appiconfg.svg" Color="#512BD4" />

    <!-- Splash Screen -->
    <MauiSplashScreen Include="Resources\Splash\splash.svg" Color="#512BD4" BaseSize="128,128" />

    <!-- Images -->
    <MauiImage Include="Resources\Images\*" />
    <MauiImage Update="Resources\Images\dotnet_bot.png" Resize="True" BaseSize="300,185" />
  </ItemGroup>

  <!-- Custom Fonts -->
  <MauiFont Include="Resources\Fonts\*" />

  <!-- Raw Assets (also remove the "Resources\Raw" prefix) -->
  <MauiAsset Include="Resources\Raw\*" LogicalName="%$(RecursiveDir)%$(Filename)%$(Extension)" />
</ItemGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.Maui.Controls" Version="$(MauiVersion)" />
    <PackageReference Include="Microsoft.Extensions.Logging.Debug" Version="10.0.0" />
  </ItemGroup>

</Project>
```

Application Resources - App.xaml

Source file: App.xaml

```
■<?xml version = "1.0" encoding = "UTF-8" ?>
<Application xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:local="clr-namespace:AklanDrugstoreInventory"
    x:Class="AklanDrugstoreInventory.App">
    <Application.Resources>
        <ResourceDictionary>
            <ResourceDictionary.MergedDictionaries>
                <ResourceDictionary Source="Resources/Styles/Colors.xaml" />
                <ResourceDictionary Source="Resources/Styles/Styles.xaml" />
            </ResourceDictionary.MergedDictionaries>
        </ResourceDictionary>
    </Application.Resources>
</Application>
```

Application Startup - App.xaml.cs

Source file: App.xaml.cs

```
using Microsoft.Extensions.DependencyInjection;

namespace AklanDrugstoreInventory;

public partial class App : Application
{
    public App()
    {
        InitializeComponent();

        protected override Window CreateWindow(IActivationState? activationState)
        {
            return new Window(new AppShell());
        }
    }
}
```

Application Shell - AppShell.xaml

Source file: AppShell.xaml

```
<?xml version="1.0" encoding="UTF-8" ?>
<Shell
  x:Class="AklanDrugstoreInventory.AppShell"
  xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
  xmlns:local="clr-namespace:Ak lanDrugstoreInventory"
  Title="Ak lan Drugstore Inventory"
  FlyoutBehavior="Disabled">

  <ShellContent
    Title="Inventory"
    ContentTemplate="{DataTemplate local:MainPage}"
    Route="MainPage" />

</Shell>
```

Application Shell Code - AppShell.xaml.cs

Source file: AppShell.xaml.cs

```
■namespace AklanDrugstoreInventory;

public partial class AppShell : Shell
{
    ■public AppShell()
    ■{
    ■■InitializeComponent();
    ■}
}
```

MAUI Configuration - MauiProgram.cs

Source file: MauiProgram.cs

```
using Microsoft.Extensions.Logging;

namespace AklanDrugstoreInventory;

public static class MauiProgram
{
    public static MauiApp CreateMauiApp()
    {
        var builder = MauiApp.CreateBuilder();
        builder
            .UseMauiApp<App>()
            .ConfigureFonts(fonts =>
            {
                fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
                fonts.AddFont("OpenSans-Semibold.ttf", "OpenSansSemibold");
            });

        #if DEBUG
        builder.Logging.AddDebug();
        #endif

        return builder.Build();
    }
}
```

Data Model - InventoryItem.cs

Source file: Models/InventoryItem.cs

```
using System.ComponentModel;
using System.Runtime.CompilerServices;

namespace AklanDrugstoreInventory.Models;

public class InventoryItem : INotifyPropertyChanged
{
    private string name = string.Empty;
    private string category = string.Empty;
    private int quantity;
    private decimal price;
    private DateTime expiryDate = DateTime.Today.AddYears(1);

    public Guid Id { get; set; }

    public string Name
    {
        get => name;
        set => SetField(ref name, value);
    }

    public string Category
    {
        get => category;
        set => SetField(ref category, value);
    }

    public int Quantity
    {
        get => quantity;
        set
        {
            if (SetField(ref quantity, value))
                OnPropertyChanged(nameof(ChartHeight));
        }
    }

    public decimal Price
    {
        get => price;
        set => SetField(ref price, value);
    }

    public DateTime ExpiryDate
    {
        get => expiryDate;
        set => SetField(ref expiryDate, value);
    }

    public double ChartHeight => Math.Clamp(Quantity * 5, 12, 160);

    public event PropertyChangedEventHandler? PropertyChanged;

    private bool SetField<T>(ref T field, T value, [CallerMemberName] string? propertyName = null)
    {
        if (EqualityComparer<T>.Default.Equals(field, value))
            return false;

        field = value;
        OnPropertyChanged(propertyName);
        return true;
    }

    private void OnPropertyChanged([CallerMemberName] string? propertyName = null) =>
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
}
```

Data Model - AuditEntry.cs

Source file: Models/AuditEntry.cs

```
namespace AklanDrugstoreInventory.Models;

public class AuditEntry
{
    public DateTime Time { get; set; }
    public string Action { get; set; } = string.Empty;
    public string Details { get; set; } = string.Empty;
}
```

User Interface - MainPage.xaml

Source file: MainPage.xaml

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:models="clr-namespace:AklanDrugstoreInventory.Models"
    x:Class="AklanDrugstoreInventory.MainPage"
    Shell.NavBarIsVisible="False"
    BackgroundColor="#F3F6F4">

    <ContentPage.Resources>
        <Style x:Key="Card" TargetType="Border">
            <Setter Property="BackgroundColor" Value="White" />
            <Setter Property="Stroke" Value="#DDE6E1" />
            <Setter Property="StrokeThickness" Value="1" />
            <Setter Property="StrokeShape" Value="RoundRectangle 14" />
            <Setter Property="Padding" Value="18" />
        </Style>
        <Style x:Key="MenuButton" TargetType="Button">
            <Setter Property="BackgroundColor" Value="Transparent" />
            <Setter Property="TextColor" Value="White" />
            <Setter Property="HorizontalOptions" Value="Fill" />
            <Setter Property="HorizontalTextAlignment" Value="Start" />
            <Setter Property="CornerRadius" Value="8" />
        </Style>
        <Style x:Key="Title" TargetType="Label">
            <Setter Property="FontFamily" Value="OpenSansSemibold" />
            <Setter Property="FontSize" Value="25" />
            <Setter Property="TextColor" Value="#15392B" />
        </Style>
        <Style x:Key="Field" TargetType="Entry">
            <Setter Property="BackgroundColor" Value="#F8FAF9" />
            <Setter Property="TextColor" Value="#19352A" />
            <Setter Property="PlaceholderColor" Value="#7B8D85" />
        </Style>
    </ContentPage.Resources>

    <Grid>
        <!-- Local login screen -->
        <Grid x:Name="LoginPanel" Padding="24">
            <Border Style="{StaticResource Card}"
                MaximumWidthRequest="430"
                HorizontalOptions="Center"
                VerticalOptions="Center">
                <VerticalStackLayout Spacing="14">
                    <Border BackgroundColor="#167A53"
                        StrokeThickness="0"
                        StrokeShape="RoundRectangle 12"
                        HeightRequest="76"
                        WidthRequest="76"
                        HorizontalOptions="Center">
                        <Label Text="Rx" TextColor="White" FontSize="30"
                            FontFamily="OpenSansSemibold"
                            HorizontalOptions="Center" VerticalOptions="Center" />
                    </Border>
                    <Label Text="Aklan Drugstore" FontSize="27"
                        FontFamily="OpenSansSemibold" TextColor="#15392B"
                        HorizontalOptions="Center" />
                    <Label Text="Inventory Management System" TextColor="#5E7169"
                        HorizontalOptions="Center" />
                    <Entry x:Name="UsernameEntry" Style="{StaticResource Field}"
                        Placeholder="Username" Text="admin" />
                    <Entry x:Name="PasswordEntry" Style="{StaticResource Field}"
                        Placeholder="Password" IsPassword="True" />
                    <Label x:Name="LoginMessage" TextColor="#B42318" IsVisible="False" />
                    <Button Text="LOG IN" BackgroundColor="#167A53" Clicked="OnLoginClicked" />
                    <Label Text="Demo account: admin / admin123" FontSize="12"
                        TextColor="#718078" HorizontalOptions="Center" />
                    <Label Text="Developed by Arjunren Valdez" FontSize="11"
                        TextColor="#8A9791" HorizontalOptions="Center" />
                </VerticalStackLayout>
            </Border>
        </Grid>

        <!-- Main application -->
        <Grid x:Name="AppPanel" IsVisible="False" ColumnDefinitions="210,*">
            <Border Grid.Column="0" BackgroundColor="#123D2E" StrokeThickness="0" Padding="16,22">
                <Grid RowDefinitions="Auto,*,Auto">
                    <VerticalStackLayout Spacing="3">
                        <Label Text="AKLAN" TextColor="#A7E0C6" FontSize="13" />
                        <Label Text="Drugstore Inventory" TextColor="White" FontSize="19"
                            FontFamily="OpenSansSemibold" />
                    </VerticalStackLayout>
                    <VerticalStackLayout Grid.Row="1" Margin="0,30,0,0" Spacing="8">
                        <Button Text="Dashboard" Style="{StaticResource MenuButton}"
                            Clicked="OnDashboardMenuClicked" />
                        <Button Text="Inventory" Style="{StaticResource MenuButton}"
                            Clicked="OnInventoryMenuClicked" />
                        <Button Text="Audit Trail" Style="{StaticResource MenuButton}"
                            Clicked="OnAuditMenuClicked" />
                    </VerticalStackLayout>
                    <VerticalStackLayout Grid.Row="2" Spacing="5">
                        <Label Text="Arjunren Valdez" TextColor="#CDE6DA" FontSize="12" />
                        <Button Text="Log out" Style="{StaticResource MenuButton}"
                            Clicked="OnLogoutClicked" />
                    </VerticalStackLayout>
                </Grid>
            </Border>

            <Grid Grid.Column="1" Padding="24">
                <!-- Dashboard and data visualization -->
                <ScrollView x:Name="DashboardPanel">
                    <VerticalStackLayout Spacing="18">
                        <Label Text="Dashboard" Style="{StaticResource Title}" />
                        <Label Text="Quick view of the drugstore inventory." TextColor="#66756F" />
                        <Grid ColumnDefinitions="*,*,*" ColumnSpacing="14">
                            <Border Style="{StaticResource Card}">
                                <VerticalStackLayout>
```

```

        <Label Text="Products" TextColor="#65746E" />
        <Label x:Name="ProductCountLabel" Text="0" FontSize="30"
            TextColor="#167A53" FontFamily="OpenSansSemibold" />
    </VerticalStackLayout>
</Border>
<Border Grid.Column="1" Style="{StaticResource Card}">
    <VerticalStackLayout>
        <Label Text="Total units" TextColor="#65746E" />
        <Label x:Name="UnitCountLabel" Text="0" FontSize="30"
            TextColor="#167A53" FontFamily="OpenSansSemibold" />
    </VerticalStackLayout>
</Border>
<Border Grid.Column="2" Style="{StaticResource Card}">
    <VerticalStackLayout>
        <Label Text="Low stock" TextColor="#65746E" />
        <Label x:Name="LowStockCountLabel" Text="0" FontSize="30"
            TextColor="#C65911" FontFamily="OpenSansSemibold" />
    </VerticalStackLayout>
</Border>
</Grid>

<Border Style="{StaticResource Card}">
    <VerticalStackLayout Spacing="14">
        <Label Text="Stock level chart" FontSize="18"
            FontFamily="OpenSansSemibold" TextColor="#234438" />
        <CollectionView x:Name="ChartView" ItemsSource="{Binding Items}"
            HeightRequest="220" ItemsLayout="HorizontalList">
            <CollectionView.ItemTemplate>
                <DataTemplate x:DataType="models:InventoryItem">
                    <Grid WidthRequest="92" RowDefinitions="*,Auto,Auto" Padding="8,0">
                        <BoxView Grid.Row="0" WidthRequest="38"
                            HeightRequest="{Binding ChartHeight}"
                            VerticalOptions="End" HorizontalOptions="Center"
                            Color="#3BA575" CornerRadius="5" />
                        <Label Grid.Row="1" Text="{Binding Quantity}" FontSize="12"
                            HorizontalOptions="Center" TextColor="#38564B" />
                        <Label Grid.Row="2" Text="{Binding Name}" FontSize="11"
                            HorizontalTextAlignment="Center" LineBreakMode="TailTruncation"
                            WidthRequest="86" TextColor="#52675F" />
                    </Grid>
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </VerticalStackLayout>
</Border>
</VerticalStackLayout>
</ScrollView>

<!-- Inventory CRUD screen -->
<Grid x:Name="InventoryPanel" IsVisible="False" RowDefinitions="Auto,*" RowSpacing="14">
    <VerticalStackLayout>
        <Label Text="Inventory Management" Style="{StaticResource Title}" />
        <Label Text="Select a row to update or delete a product." TextColor="#66756F" />
    </VerticalStackLayout>
    <Grid Grid.Row="1" ColumnDefinitions="320,*" ColumnSpacing="16">
        <Border Style="{StaticResource Card}">
            <ScrollView>
                <VerticalStackLayout Spacing="10">
                    <Label Text="Product details" FontSize="18" FontFamily="OpenSansSemibold"
                        TextColor="#234438" />
                    <Entry x:Name="NameEntry" Style="{StaticResource Field}" Placeholder="Product name" />
                    <Entry x:Name="CategoryEntry" Style="{StaticResource Field}" Placeholder="Category" />
                    <Entry x:Name="QuantityEntry" Style="{StaticResource Field}" Placeholder="Quantity"
                        Keyboard="Numeric" />
                    <Entry x:Name="PriceEntry" Style="{StaticResource Field}" Placeholder="Price"
                        Keyboard="Numeric" />
                    <DatePicker x:Name="ExpiryPicker" MinimumDate="2024-01-01" />
                    <Label x:Name="FormMessage" FontSize="12" TextColor="#B42318" IsVisible="False" />
                    <Grid ColumnDefinitions="*,*" ColumnSpacing="8">
                        <Button Text="Add" BackgroundColor="#167A53" Clicked="OnAddClicked" />
                        <Button Grid.Column="1" Text="Update" BackgroundColor="#2F6F9F" Clicked="OnUpdateClicked" />
                    </Grid>
                    <Grid ColumnDefinitions="*,*" ColumnSpacing="8">
                        <Button Text="Delete" BackgroundColor="#B33A3A" Clicked="OnDeleteClicked" />
                        <Button Grid.Column="1" Text="Clear" BackgroundColor="#687873" Clicked="OnClearClicked" />
                    </Grid>
                </VerticalStackLayout>
            </ScrollView>
        </Border>

        <Border Grid.Column="1" Style="{StaticResource Card}">
            <CollectionView x:Name="InventoryView" ItemsSource="{Binding Items}"
                SelectionMode="Single" SelectionChanged="OnInventorySelectionChanged">
                <CollectionView.Header>
                    <Grid ColumnDefinitions="2*,1.2*,.7*,.8*,1.1*" Padding="8,6" BackgroundColor="#E8F2ED">
                        <Label Text="Product" FontFamily="OpenSansSemibold" />
                        <Label Grid.Column="1" Text="Category" FontFamily="OpenSansSemibold" />
                        <Label Grid.Column="2" Text="Qty" FontFamily="OpenSansSemibold" />
                        <Label Grid.Column="3" Text="Price" FontFamily="OpenSansSemibold" />
                        <Label Grid.Column="4" Text="Expiry" FontFamily="OpenSansSemibold" />
                    </Grid>
                </CollectionView.Header>
                <CollectionView.ItemTemplate>
                    <DataTemplate x:DataType="models:InventoryItem">
                        <Grid ColumnDefinitions="2*,1.2*,.7*,.8*,1.1*" Padding="8,10">
                            <Label Text="{Binding Name}" LineBreakMode="TailTruncation" />
                            <Label Grid.Column="1" Text="{Binding Category}" LineBreakMode="TailTruncation" />
                            <Label Grid.Column="2" Text="{Binding Quantity}" />
                            <Label Grid.Column="3" Text="{Binding Price, StringFormat='{0:F2}'}" />
                            <Label Grid.Column="4" Text="{Binding ExpiryDate, StringFormat='{0:MM dd, yyyy}'}" />
                        </Grid>
                    </DataTemplate>
                </CollectionView.ItemTemplate>
                <CollectionView.EmptyView>
                    <Label Text="No inventory records yet." HorizontalOptions="Center"
                        VerticalOptions="Center" TextColor="#718078" />
                </CollectionView.EmptyView>
            </CollectionView>
        </Border>
    </Grid>
</Grid>

<!-- Audit trail -->
<Grid x:Name="AuditPanel" IsVisible="False" RowDefinitions="Auto,*" RowSpacing="14">

```

```

<VerticalStackLayout>
  <Label Text="Audit Trail" Style="{StaticResource Title}" />
  <Label Text="Records of login and inventory transactions." TextColor="#66756F" />
</VerticalStackLayout>
<Border Grid.Row="1" Style="{StaticResource Card}">
  <CollectionView ItemsSource="{Binding AuditEntries}">
    <CollectionView.ItemTemplate>
      <DataTemplate x:DataType="models:AuditEntry">
        <Grid ColumnDefinitions="160,120,*" Padding="8,11">
          <Label Text="{Binding Time, StringFormat='{0:MMM dd, yyyy HH:mm}'}" FontSize="12" />
          <Label Grid.Column="1" Text="{Binding Action}" FontFamily="OpenSansSemibold"
            TextColor="#167A53" />
          <Label Grid.Column="2" Text="{Binding Details}" LineBreakMode="WordWrap" />
        </Grid>
      </DataTemplate>
    </CollectionView.ItemTemplate>
  </CollectionView>
</Border>
</Grid>
</Grid>
</Grid>
</Grid>
</ContentPage>

```

Application Logic - MainPage.xaml.cs

Source file: MainPage.xaml.cs

```
using System.Collections.ObjectModel;
using System.Text.Json;
using AklanDrugstoreInventory.Models;

namespace AklanDrugstoreInventory;

public partial class MainPage : ContentPage
{
    private readonly string inventoryFile = Path.Combine(FileSystem.AppDataDirectory, "inventory.json");
    private readonly string auditFile = Path.Combine(FileSystem.AppDataDirectory, "audit.json");
    private InventoryItem? selectedItem;
    private bool dataLoaded;

    public ObservableCollection<InventoryItem> Items { get; } = [];
    public ObservableCollection<AuditEntry> AuditEntries { get; } = [];

    public MainPage()
    {
        InitializeComponent();
        BindingContext = this;
        ExpiryPicker.Date = DateTime.Today.AddYears(1);
    }

    protected override async void OnAppearing()
    {
        base.OnAppearing();
        if (dataLoaded)
            return;

        dataLoaded = true;
        await LoadDataAsync();
        UpdateDashboard();
    }

    private void OnLoginClicked(object? sender, EventArgs e)
    {
        if (UsernameEntry.Text?.Trim().Equals("admin", StringComparison.OrdinalIgnoreCase) == true &&
            PasswordEntry.Text == "admin123")
        {
            LoginMessage.IsVisible = false;
            LoginPanel.IsVisible = false;
            AppPanel.IsVisible = true;
            AddAudit("LOGIN", "Local administrator logged in.");
            ShowPanel(DashboardPanel);
            return;
        }

        LoginMessage.Text = "Incorrect username or password.";
        LoginMessage.IsVisible = true;
    }

    private void OnLogoutClicked(object? sender, EventArgs e)
    {
        AddAudit("LOGOUT", "Local administrator logged out.");
        PasswordEntry.Text = string.Empty;
        AppPanel.IsVisible = false;
        LoginPanel.IsVisible = true;
    }

    private void OnDashboardMenuClicked(object? sender, EventArgs e) => ShowPanel(DashboardPanel);
    private void OnInventoryMenuClicked(object? sender, EventArgs e) => ShowPanel(InventoryPanel);
    private void OnAuditMenuClicked(object? sender, EventArgs e) => ShowPanel(AuditPanel);

    private void ShowPanel(VisualElement panel)
    {
        DashboardPanel.IsVisible = panel == DashboardPanel;
        InventoryPanel.IsVisible = panel == InventoryPanel;
        AuditPanel.IsVisible = panel == AuditPanel;
        UpdateDashboard();
    }

    private async void OnAddClicked(object? sender, EventArgs e)
    {
        if (!TryReadForm(out string name, out string category, out int quantity, out decimal price))
            return;

        var item = new InventoryItem
        {
            Id = Guid.NewGuid(),
            Name = name,
            Category = category,
            Quantity = quantity,
            Price = price,
            ExpiryDate = ExpiryPicker.Date ?? DateTime.Today
        };

        Items.Add(item);
        AddAudit("ADD", $"Added {item.Name} with {item.Quantity} unit(s).");
        await SaveDataAsync();
        ClearForm();
        UpdateDashboard();
    }

    private async void OnUpdateClicked(object? sender, EventArgs e)
    {
        if (selectedItem is null)
        {
            ShowFormError("Select a product to update.");
            return;
        }

        if (!TryReadForm(out string name, out string category, out int quantity, out decimal price))
            return;

        string oldName = selectedItem.Name;
        selectedItem.Name = name;
```

```

        selectedItem.Category = category;
        selectedItem.Quantity = quantity;
        selectedItem.Price = price;
        selectedItem.ExpiryDate = ExpiryPicker.Date ?? DateTime.Today;

        AddAudit("UPDATE", $"Updated {oldName} to {selectedItem.Name} ({selectedItem.Quantity} unit(s)).");
        await SaveDataAsync();
        ClearForm();
        UpdateDashboard();
    }

    private async void OnDeleteClicked(object? sender, EventArgs e)
    {
        if (selectedItem is null)
        {
            ShowFormError("Select a product to delete.");
            return;
        }

        bool confirmed = await DisplayAlertAsync("Delete product", $"Remove {selectedItem.Name}?", "Delete", "Cancel");
        if (!confirmed)
            return;

        string productName = selectedItem.Name;
        Items.Remove(selectedItem);
        AddAudit("DELETE", $"Deleted {productName}.");
        await SaveDataAsync();
        ClearForm();
        UpdateDashboard();
    }

    private void OnClearClicked(object? sender, EventArgs e) => ClearForm();

    private void OnInventorySelectionChanged(object? sender, SelectionChangedEventArgs e)
    {
        selectedItem = e.CurrentSelection.FirstOrDefault() as InventoryItem;
        if (selectedItem is null)
            return;

        NameEntry.Text = selectedItem.Name;
        CategoryEntry.Text = selectedItem.Category;
        QuantityEntry.Text = selectedItem.Quantity.ToString();
        PriceEntry.Text = selectedItem.Price.ToString("0.00");
        ExpiryPicker.Date = selectedItem.ExpiryDate;
        FormMessage.IsVisible = false;
    }

    private bool TryReadForm(out string name, out string category, out int quantity, out decimal price)
    {
        name = NameEntry.Text?.Trim() ?? string.Empty;
        category = CategoryEntry.Text?.Trim() ?? string.Empty;
        bool quantityOk = int.TryParse(QuantityEntry.Text, out quantity) && quantity >= 0;
        bool priceOk = decimal.TryParse(PriceEntry.Text, out price) && price >= 0;

        if (string.IsNullOrWhiteSpace(name) || string.IsNullOrWhiteSpace(category) || !quantityOk || !priceOk)
        {
            ShowFormError("Enter a name, category, valid quantity, and valid price.");
            return false;
        }

        FormMessage.IsVisible = false;
        return true;
    }

    private void ShowFormError(string message)
    {
        FormMessage.Text = message;
        FormMessage.IsVisible = true;
    }

    private void ClearForm()
    {
        selectedItem = null;
        InventoryView.SelectedItem = null;
        NameEntry.Text = string.Empty;
        CategoryEntry.Text = string.Empty;
        QuantityEntry.Text = string.Empty;
        PriceEntry.Text = string.Empty;
        ExpiryPicker.Date = DateTime.Today.AddYears(1);
        FormMessage.IsVisible = false;
    }

    private void UpdateDashboard()
    {
        ProductCountLabel.Text = Items.Count.ToString();
        UnitCountLabel.Text = Items.Sum(item => item.Quantity).ToString();
        LowStockCountLabel.Text = Items.Count(item => item.Quantity <= 10).ToString();
    }

    private void AddAudit(string action, string details)
    {
        AuditEntries.Insert(0, new AuditEntry
        {
            Time = DateTime.Now,
            Action = action,
            Details = details
        });

        _ = SaveAuditAsync();
    }

    private async Task LoadDataAsync()
    {
        try
        {
            if (File.Exists(inventoryFile))
            {
                string json = await File.ReadAllTextAsync(inventoryFile);
                foreach (InventoryItem item in JsonSerializer.Deserialize<List<InventoryItem>>(json) ?? [])
                    Items.Add(item);
            }

            if (File.Exists(auditFile))
            {

```

```

        string json = await File.ReadAllTextAsync(auditFile);
        foreach (AuditEntry entry in JsonSerializer.Deserialize<List<AuditEntry>>(json) ?? [])
            AuditEntries.Add(entry);
    }
}
catch
{
    // A damaged local file should not prevent the student demo from opening.
}

if (Items.Count == 0)
{
    Items.Add(new InventoryItem { Id = Guid.NewGuid(), Name = "Paracetamol 500mg", Category = "Medicine", Quantity = 25, Price = 5.50m, ExpiryDate = DateTime.Today.AddYears(1) });
    Items.Add(new InventoryItem { Id = Guid.NewGuid(), Name = "Vitamin C", Category = "Vitamins", Quantity = 18, Price = 8.00m, ExpiryDate = DateTime.Today.AddYears(1) });
    Items.Add(new InventoryItem { Id = Guid.NewGuid(), Name = "Alcohol 70%", Category = "First Aid", Quantity = 9, Price = 45.00m, ExpiryDate = DateTime.Today.AddMonths(12) });
    Items.Add(new InventoryItem { Id = Guid.NewGuid(), Name = "Face Mask", Category = "Supplies", Quantity = 32, Price = 3.00m, ExpiryDate = DateTime.Today.AddYears(1) });
    await SaveDataAsync();
}

private async Task SaveDataAsync()
{
    string json = JsonSerializer.Serialize(Items, new JsonSerializerOptions { WriteIndented = true });
    await File.WriteAllTextAsync(inventoryFile, json);
}

private async Task SaveAuditAsync()
{
    string json = JsonSerializer.Serialize(AuditEntries, new JsonSerializerOptions { WriteIndented = true });
    await File.WriteAllTextAsync(auditFile, json);
}
}

```

5. Credits and License

Designed and developed by **Arjunren Valdez**.

This project is released under the MIT License. The repository includes the full LICENSE file, README, source code, assets, and this documentation.